Step 8: **In com.jsp.controller package , create a controller class annotated with @Controller annotation.**

In controller class , we will be creating request handler methods. These handler methods are methods annotated with annotations like @RequestMapping etc and have specific URLs.

These methods handle incoming HTTP requests, process them, and return a response.

In controller class , here we create a method which returns view name (filename) as a response.

```
@Controller
public class DemoController {

    @RequestMapping("/hi")
    public String get()
    {
        return "index";
    }
}
}
```

URL to send request to this request handler method.

Step 9 : Whenever user will give the URL as /hi , request will be sent from DispatcherServlet to this method. This method will accept the request , and as a response , we are returning index file to the user.

So create index.jsp file in views folder and write whatever data you want to display on the browser.

- **When a controller returns a view name, the DispatcherServlet uses the configured ViewResolver to convert that name into the actual view file. Since we have configured the view Resolver , It will look in the /WEB-INF/views/ folder (as per the prefix) and adds .jsp (as per the suffix) to find the file and if file is present , it returns it to the user as a view.**

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1>Hello , this is my first MVC project</h1>
</body>
</html>
```

Step 10: Now run the project , right click on project name → run as → run on server → select tomcat server → finish.

Once the server starts , it will launch the browser and you will get 404 error.

Specify the URL in the URL bar : http://localhost:8080/springMvcDemo/hi

NOTE : In controller class , multiple request handler methods (i.e.. Multiple mappings) can be created where each handler method can handle different types of requests..

Whenever we create a handler method , these methods must always return view (the page that needs to be displayed on the browser).

To bind data using model (to send data from controller to JSP file) :

- the handler methods in controller will always return view.
- From controller , along with returning the JSP page as view , we can bind data using model and send it to JSP page.

- All the steps from 1 to 6 are same.

Step 7 : In controller class, we will create a handler method and bind data using model.

The method then returns view name i.e..JSP file name as response.(The model also gets transferred to the JSP page, where we can access the model data).

```
@Controller
public class DemoController {

    @RequestMapping("/hello")
    public String sayHello(Model model) {
        model.addAttribute("myname", "Sheela");
        return "welcome";
    }

}
```

Declare a Model type variable for Binding(storing) data. Holds data in the form of key-value pairs.
This data is accessible in the view.

addAttribute(String attributeName , Object attribute value)

Attribute name and value/object is added to the model using addAttribute() method.
Attribute name will be used in JSP page for accessinig the model data.

step 9 : Whenever user will give the URL as /hello , DispatcherServlet will send the request to sayHello handler method which will accept the request.

It will the add the specified data/object into the model and return welcome file name as response . View resolver will then resolve the file name into actual file and return welcome.jsp page.

So we will now create welcome.jsp page.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8" isELIgnored="false" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

<h1>WELCOME TO MY PAGE</h1>
<h1>MY NAME IS ${myname}</h1>

</body>
</html>
```

To evaluate the expression

Here , we will write whatever data we want to display to the user.
To access the model data sent from the controller , we use ${ attributeName} .
${attributeName} is an expression. If we want this expression to be evaluated , then we need to declare isELIgnored = "false" .
Otherwise , it will be treated as plain text and will be ignored.